

Spring Boot Configurati

`application.properties`

and Profiles

1. Why Configuration Is Needed

A real-world application is not just code. It is a combination of two things:

```
Application = Code + Configuration
```

Code contains the actual business logic.

Examples:

```
Create student
Fetch student
Update student
Delete student
Connect to database
Send email
Run server
```

But to run this code properly, the application also needs some values.

Examples:

```
Which port should the application run on?
Which database should it connect to?
What is the database username?
What is the database password?
What should be the application name?
Which environment is currently active?
What message should be shown to the user?
```

These values are called **configuration values**.

In real projects, the same Spring Boot application may run in environments:

```
Local machine
Testing server
Staging server
Production server
```

The Java code usually remains the same, but configuration values change from one environment to another.

2. The Problem With Hardcoding Values

Hardcoding means writing fixed values directly inside Java classes.

Example:

```
String dbUrl = "jdbc:mysql://localhost:3306/student_db";
String username = "root";
String password = "root";
```

At first, this may look simple. But it creates serious problems in real-world applications.

Problem 1: Code Becomes Difficult to Change

Suppose the database URL changes.

If the value is hardcoded inside Java code, we must open the Java file, update the value, rebuild the application, and redeploy it.

This is unnecessary because database URL is not business logic. It is an environment-specific configuration value.

Problem 2: Same Code Cannot Easily Run in Multiple Environments

Your local machine may use:

```
username = root
password = root
```

Production may use:

```
username = prod_user
password = strong_secure_password
```

If these values are hardcoded, the same code becomes tightly attached to one environment.

A professional application should follow this idea:

```
Same code
Different configuration
```

Problem 3: Sensitive Information Can Leak

If you write passwords directly inside Java code:

```
String password = "admin123";
```

and then push the code to GitHub, the password may get exposed.

This is why sensitive values like passwords, API keys, and tokens should not be hardcoded inside Java classes.

3. What Is **application.properties** ?

Spring Boot provides a special configuration file:

```
application.properties
```

This file is used to write application configuration values in one central place.

Example:

```
server.port=8081
spring.application.name=student-management-app
app.welcome.message=Welcome to Student Management App
```

Instead of writing these values inside Java code, we write them in `application.properties`.

4. Why Is It Named `application.properties` ?

Spring Boot automatically looks for a file named:

```
application.properties
```

during application startup.

When the application starts, Spring Boot checks whether this file exists. If it exists, Spring Boot loads the values from it.

So `application.properties` becomes the central place for application configuration.

5. Basic Key-Value Format in `application.properties`

The format is simple:

```
key=value
```

Example:

```
server.port=8081
spring.application.name=student-management-app
app.owner=
```

Here:

server.port	→ key
8081	→ value

```
spring.application.name    → key
student-management-app    → value
```

6. Reading Configuration Values Using @

Spring provides the `@Value` annotation to inject configuration variables.

Example:

```
app.welcome.message=Welcome to Student Management App
```

Java code:

```
@RestController
public class WelcomeController {

    @Value("${app.welcome.message}")
    private String welcomeMessage;

    @GetMapping("/welcome")
    public String getWelcomeMessage() {
        return welcomeMessage;
    }
}
```

Here:

```
@Value("${app.welcome.message}")
```

means:

```
Read the value of app.welcome.message from the configuration file
and inject it into this variable.
```

This is called **property injection**.

7. Limitations of `application.properties`

For small projects, `application.properties` is simple and readable.

Example:

```
server.port=8081
spring.application.name=student-management-app
app.welcome.message=Welcome to Student Management App
```

But as the project grows, the file can become large and repetitive.

7.1 Flat Structure

`application.properties` stores values in a flat key-value format.

Example:

```
app.name=Student Management App
app.version=1.0
app.owner=
app.support.email=support@studentapp.com
app.support.phone=9999999999
```

The values are related, but the file does not visually show a proper hierarchy.

7.2 Repeated Prefixes

For grouped configuration, prefixes are repeated again and again.

Example:

```
notification.email.enabled=true
notification.email.subject=Welcome
notification.email.from=no-reply@studentapp.com
notification.email.retry.count=3

notification.sms.enabled=false
```

```
notification.sms.provider=twilio
notification.sms.retry.count=2
```

This works, but it becomes less readable when configuration

7.3 Lists Are Not Very Clean

Example:

```
app.supported-cities=Delhi,Mumbai,Bangalore,Pune
```

This is fine for a small list, but for bigger lists or nested objects, `.properties` becomes difficult to read.

8. Introduction to YAML

YAML solves the readability problem of large configuration files.

The problem is not that `application.properties` is wrong. The problem is that `.properties` becomes flat and repetitive when configuration grows.

Example in `.properties`:

```
app.name=Student Management App
app.version=1.0
app.owner=
app.support.email=support@studentapp.com
app.support.phone=9999999999
```

Actually, this data has a natural hierarchy:

```
app
├── name
├── version
├── owner
└── support
    ├── email
    └── phone
```

YAML allows us to write this in a structured way.

Same configuration in YAML:

```
app:
  name: Student Management App
  version: 1.0
  owner: 
  support:
  email: support@studentapp.com
  phone: 9999999999
```

YAML converts flat configuration into structured configuration.

```
.properties → flat key-value format
.yml         → structured, nested format
```

9. How Spring Boot Understands YAML

YAML looks nested, but internally Spring Boot understands it as normal property keys.

Example YAML:

```
server:
  port: 8081
```

Spring Boot understands it like:

```
server.port=8081
```

Another example:

```
spring:
  application:
```

```
name: student-management-app
```

Spring Boot understands it like:

```
spring.application.name=student-management-app
```

So YAML is not a completely different configuration system. I write the same configuration.

10. `application.properties` vs `application.yml`

Spring Boot supports both formats:

```
application.properties
application.yml
application.yml
```

But for a project, it is better to stick to one format.

Avoid keeping both `application.properties` and `application.yml` for the same purpose in the same location, because it can create confusion about which value is actually being used.

A clean project should usually follow one consistent style.

11. YAML Syntax and Indentation Rules

YAML is simple, but it is strict about indentation.

In `.properties`:

```
server.port=8081
```

In YAML:

```
server:
```

```
port:8081
```

Here:

```
server → parent  
port   → child
```

The spaces before `port` show that `port` belongs inside `server`.

Rule 1: Use Colon :

Every key is followed by a colon.

```
app:  
name: Student Management App
```

Rule 2: Put a Space After the Colon

Correct:

```
server:  
port:8081
```

Avoid:

```
server:  
port:8081
```

Use:

```
key: value
```

not:

```
key:value
```

Rule 3: Indentation Defines Relationship

Correct:

```
app:
  support:
    email: support@studentapp.com
```

This means:

```
app.support.email=support@studentapp.com
```

Wrong:

```
app:
support:
email: support@studentapp.com
```

This destroys the hierarchy.

YAML does not use dots like `.properties`. YAML uses spaces to show structure.

Rule 4: Use Spaces, Not Tabs

YAML is indentation-sensitive.

Use spaces only. Usually, two spaces are used.

Correct:

```
spring:
  application:
    name: student-management-app
```

Avoid tabs.

Rule 5: Same-Level Keys Must Have Same Indentation

Correct:

```
app:
  name: Student Management App
  version: 1.0
  owner: 
```

Here, `name`, `version`, and `owner` are at the same level.

Rule 6: Comments Use `#`

```
# Server configuration
server:
  port: 8081
```

Comments are useful for dividing configuration sections.

Rule 7: Strings Can Be Written With or Without Quotes

Both are usually valid:

```
app:
  name: Student Management App
```

and:

```
app:
  name: "Student Management App"
```

Quotes are useful when the value contains special characters, placeholders, or symbols.

12. Converting `application.properties` to `application.yml`

Example `application.properties`:

```
server.port=8081

spring.application.name=student-management-app

app.name=Student Management App
app.version=1.0
app.owner=

app.welcome.message=Welcome to Student Management App

app.support.email=support@studentapp.com
app.support.phone=9999999999
```

Equivalent `application.yml` :

```
server:
  port: 8081

spring:
  application:
    name: student-management-app

  app:
    name: Student Management App
    version: 1.0
    owner: 
    welcome:
      message: Welcome to Student Management App
    support:
      email: support@studentapp.com
      phone: 9999999999
```

13. Real-World Example: Database Configuration

In `application.properties` :

```
spring.datasource.url=jdbc:mysql://localhost:3306/student_db
spring.datasource.username=root
```

```
spring.datasource.password=root
```

In `application.yml`:

```
spring:
  datasource:
    url: jdbc:mysql://localhost:3306/student_db
    username: root
    password: root
```

Now all database-related configuration is grouped under:

```
spring.datasource
```

14. Real-World Example: JPA Configuration

In `application.properties`:

```
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
```

In `application.yml`:

```
spring:
  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
```

This is more readable because related configuration is grouped together.

15. Lists and Multiple Values in YAML

Suppose one key has multiple values.

Example:

```
Supported cities:  
Delhi  
Mumbai  
Bangalore  
Pune
```

In `.properties`, we may write:

```
app.supported-cities=Delhi,Mumbai,Bangalore,Pune
```

This works, but it looks like one long string.

In YAML, we can write it as a proper list:

```
app:  
  supported-cities:  
    - Delhi  
    - Mumbai  
    - Bangalore  
    - Pune
```

The `-` symbol represents one item in the list.

This:

```
supported-cities:  
  - Delhi  
  - Mumbai  
  - Bangalore
```

means:

```
supported-cities has multiple values:  
1. Delhi
```

2. Mumbai
3. Bangalore

16. List of Objects in YAML

YAML can also represent a list of objects.

Example:

```
app:
  team-members:
    -name: Rohit
    role: Backend Mentor
    -name: Mohit
    role: DSA Mentor
    -name: Aditya
    role: Spring Mentor
```

This type of structure becomes difficult to manage in `.properties`.

YAML makes it much more readable.

For complex nested configuration, Spring Boot also provides:

```
@ConfigurationProperties
```

`@ConfigurationProperties` is better when we need to bind multiple related values, lists, or nested objects into a Java class.

17. Reading YAML Values Using Dot Notation

Even though YAML is nested, we still read values using dot notation.

YAML:

```
app:
  name: Student Management App
```

Java:

```
@Value("${app.name}")  
private String appName;
```

Spring internally treats it like:

```
app.name=Student Management App
```

18. Reading List Values

For beginner-level teaching, list values can be shown in two simple ways.

Way 1: Reading Individual List Items

`application.yml`:

```
app:  
  supported-cities:  
    - Delhi  
    - Mumbai  
    - Bangalore
```

Java:

```
@Value("${app.supported-cities[0]}")  
private String firstCity;  
  
@Value("${app.supported-cities[1]}")  
private String secondCity;
```

This reads:

```
Delhi
```

```
Mumbai
```

This shows that YAML lists are index-based internally.

Way 2: Reading Comma-Separated Values as a List

For `@Value`, this style is easier:

```
app:
  supported-cities: Delhi,Mumbai,Bangalore,Pune
```

Java:

```
@Value("#{ '${app.supported-cities}'.split(',') }")
private List<String> supportedCities;
```

This converts the comma-separated string into a Java `List<String>`.

For professional projects, avoid using too many `@Value` annotations for complex configuration. Prefer `@ConfigurationProperties`.

19. Why Spring Profiles Are Needed

Till now, we solved one problem:

```
Configuration values should not be hardcoded inside Java code.
```

So we moved values into:

```
application.properties
application.yml
```

Now the next real-world problem is:

```
The same application needs different values in different environments.
```

That is where **Spring Profiles** come in.

Spring Boot allows the same application code to work with different sources such as:

```
Properties files
YAML files
Environment variables
Command-line arguments
```

20. Problem With Multiple Environments

In real projects, a Spring Boot application is not used only on a developer laptop.

It may run in:

```
Development environment
Testing environment
Staging environment
Production environment
```

The Java logic usually remains the same:

```
Create student
Fetch student
Update student
Delete student
Validate request
Handle exception
Connect to database
```

But the surrounding values change.

Example:

```
Local machine:
Database URL = jdbc:mysql://localhost:3306/student_db
```

```
Production server:  
Database URL = jdbc:mysql://prod-db-server:3306/student_db
```

The application logic is the same, but the database location i

This is the main idea:

```
Same code  
Different configuration
```

21. Example Without Profiles

Suppose this is your local configuration:

```
server:  
port: 8080  
  
spring:  
datasource:  
url: jdbc:mysql://localhost:3306/student_db  
username: root  
password: root
```

For production, you may need:

```
server:  
port: 8080  
  
spring:  
datasource:  
url: jdbc:mysql://prod-server:3306/student_db  
username: prod_user  
password: prod_password
```

A bad approach is:

```
Comment local configuration
Uncomment production configuration
Run application
Again comment production configuration
Uncomment local configuration
```

This is risky.

One wrong uncommented line can connect your local application to the production database.

So we need a clean mechanism to separate environment-specific configuration.

22. Introduction to Spring Profiles

A **profile** is a named environment or mode in which the application runs.

Examples:

```
dev
test
staging
prod
```

Simple meaning:

```
Profile = Which environment configuration should Spring Boot use right now?
```

Examples:

```
If dev profile is active:
Use development configuration.

If prod profile is active:
Use production configuration.
```

Instead of changing code again and again, we tell Spring Boot:

```
Run this application with the dev profile.
```

or:

```
Run this application with the prod profile.
```

Then Spring Boot loads the matching configuration.

23. Creating Profile-Specific Files

Spring Boot supports profile-specific configuration files using this naming pattern:

```
application-{profile}.properties
```

Examples:

```
application-dev.properties  
application-test.properties  
application-prod.properties
```

Project structure:

```
src/main/resources  
├─ application.properties  
├─ application-dev.properties  
├─ application-test.properties  
└─ application-prod.properties
```

24. Meaning of Each File

application.properties

This is the common/default file.

Put values here that are common for all environments.

```
spring.application.name=student-management-app
```

application-dev.properties

This is for development.

```
server.port=8081  
app.message=Application is running in DEV mode
```

application-test.properties

This is for testing.

```
server.port=8082  
app.message=Application is running in TEST mode
```

application-prod.properties

This is for production.

```
server.port=8080  
app.message=Application is running in PROD mode
```

25. Common File + Profile-Specific File

Spring Boot does not simply ignore the common file.

If the **dev** profile is active, Spring Boot loads:

```
application.properties  
+  
application-dev.properties
```

Example:

`application.properties`

```
spring.application.name=student-management-app
```

`application-dev.properties`

```
server.port=8081
```

Final configuration:

```
spring.application.name=student-management-app  
server.port=8081
```

So the common file provides shared values, and the profile-specific file provides environment-specific values.

26. What If the Same Key Exists in Both Files?

Example:

`application.properties`

```
server.port=8080  
app.message=Common message
```

`application-dev.properties`

```
server.port=8081  
app.message=DEV message
```

If the `dev` profile is active, final values become:

```
server.port=8081  
app.message=DEV message
```

Profile-specific configuration overrides common configuration.

This is an important point for students:

```
Common file gives default values.  
Profile-specific file can override those values.
```

27. Profile-Specific YAML Files

The same concept works with YAML.

Instead of:

```
application-dev.properties  
application-prod.properties
```

we can use:

```
application-dev.yml  
application-prod.yml
```

or:

```
application-dev.yaml  
application-prod.yaml
```

Project structure:

```
src/main/resources  
├─ application.yml  
├─ application-dev.yml  
├─ application-test.yml  
└─ application-prod.yml
```

28. Common YAML File: `application.yml`

```
spring:
  application:
    name: student-management-app
```

This value is common for all profiles.

29. Dev YAML File: `application-dev.yml`

```
server:
  port: 8081

app:
  message: Application is running in DEV mode

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/student_dev
    username: root
    password: root
```

30. Test YAML File: `application-test.yml`

```
server:
  port: 8082

app:
  message: Application is running in TEST mode

spring:
  datasource:
    url: jdbc:mysql://localhost:3306/student_test
```

```
username: test_user
password: test_password
```

31. Prod YAML File: `application-prod.yml`

```
server:
  port: 8080

app:
  message: Application is running in PROD mode

spring:
  datasource:
    url: jdbc:mysql://prod-server:3306/student_prod
    username: prod_user
    password: prod_password
```

32. Activating a Profile

Creating profile-specific files is not enough.

We also need to tell Spring Boot which profile should be active.

Example:

```
Use dev profile
Use prod profile
Use test profile
```

There are multiple ways to activate a profile.

33. Activating Profile Using `application.properties`

Inside `application.properties`:

```
spring.application.name=student-management-app
spring.profiles.active=dev
```

Project structure:

```
src/main/resources
├─ application.properties
├─ application-dev.properties
└─ application-prod.properties
```

`application-dev.properties` :

```
server.port=8081
app.message=Application is running in DEV mode
```

`application-prod.properties` :

```
server.port=8080
app.message=Application is running in PROD mode
```

When the application starts, Spring Boot sees:

```
spring.profiles.active=dev
```

So it loads the dev-specific configuration.

34. Controller Demo for Profile Message

```
@RestController
public class ProfileDemoController {

    @Value("${app.message}")
    private String message;
```

```
@GetMapping("/profile-message")
public String getProfileMessage() {
    return message;
}
```

If `dev` profile is active, response:

```
Application is running in DEV mode
```

If `prod` profile is active, response:

```
Application is running in PROD mode
```

Same controller. Same Java code. Different output.

The output changes because the active profile changes.

35. Important Production Note

For beginner demos, activating the profile inside `application.properties` or `application.yml` is okay.

But in real production deployment, we usually avoid hardcoding this inside the packaged application:

```
spring:
  profiles:
    active: prod
```

Reason:

```
The same JAR should be able to run in different environments.
```

Better production-style approach:

Pass the active profile from outside.

Examples:

```
Command line
Environment variable
Docker configuration
Kubernetes configuration
Server configuration
CI/CD pipeline
```

This keeps the application artifact reusable.

36. Activating Profile From Command Line

When we build a Spring Boot JAR, we can run it like this:

```
java -jar student-management-app.jar --spring.profiles.active=dev
```

For production:

```
java -jar student-management-app.jar --spring.profiles.active=prod
```

This method is powerful because the same JAR can run differently.

```
Same JAR + dev profile = Development behavior
Same JAR + prod profile = Production behavior
```

This is professional because:

```
Code is same.
Build is same.
Environment decides configuration.
```

37. Activating Profile While Running With

During development, if you are running the application with

```
mvn spring-boot:run -Dspring-boot.run.profiles
```

This starts the application with the `dev` profile active.

38. Activating Profile Using Environment Variables

Another professional way is using environment variables.

For Spring profiles, the environment variable name is:

```
SPRING_PROFILES_ACTIVE
```

macOS/Linux:

```
export SPRING_PROFILES_ACTIVE=dev  
java -jar student-management-app.jar
```

Windows PowerShell:

```
$env:SPRING_PROFILES_ACTIVE="dev"  
java -jar student-management-app.jar
```

39. Why Environment Variables Are Used

In production, values often come from the server or deployment platform.

Examples:

```
Docker container environment variable  
Kubernetes environment variable  
Cloud platform environment variable  
CI/CD pipeline variable  
Server-level configuration
```

Instead of putting this inside the project:

```
spring:
  profiles:
    active: prod
```

production can provide:

```
SPRING_PROFILES_ACTIVE=prod
```

This is cleaner and safer because the environment decides which profile should run.

40. Practical Demo: Different Server Ports for Different Profiles

Project structure:

```
src/main/resources
├─ application.yml
├─ application-dev.yml
└─ application-prod.yml
```

`application.yml`:

```
spring:
  application:
    name: student-management-app
```

`application-dev.yml`:

```
server:
  port: 8081
```

```
app:
message: Running in DEV profile
```

```
application-prod.yml:
```

```
server:
port: 9090

app:
message: Running in PROD profile
```

Run with dev:

```
java -jar student-management-app.jar --spring.profiles.active
=dev
```

Application runs on:

```
http://localhost:8081
```

Run with prod:

```
java -jar student-management-app.jar --spring.profiles.active
=prod
```

Application runs on:

```
http://localhost:9090
```

Same code. Same JAR. Different ports.

This is a simple demo that helps students clearly see the use of profiles.

41. Practical Demo: Different Custom Me Different Profiles

`application-dev.yml` :

```
app:
message: Hello from DEV environment
```

`application-prod.yml` :

```
app:
message: Hello from PROD environment
```

Controller:

```
@RestController
@RequestMapping("/api/profile")
public class ProfileController {

    @Value("${app.message}")
    private String message;

    @GetMapping("/message")
    public String getMessage() {
        return message;
    }
}
```

Run with dev:

```
java -jar student-management-app.jar --spring.profiles.active=dev
```

Hit:

```
GET /api/profile/message
```

Response:

```
Hello from DEV environment
```

Run with prod:

```
java -jar student-management-app.jar --spring.profiles.active=prod
```

Response:

```
Hello from PROD environment
```

Important observation:

```
The controller did not change.  
The endpoint did not change.  
Only the active profile changed.
```

42. `@Profile` Annotation and Conditional Beans

Till now, profiles were mainly used for changing configuration values.

Examples:

```
dev profile → different port, different message, different database  
prod profile → different port, different message, different database
```

Now we move one step deeper.

Sometimes we do not only want different values. Sometimes we want different Spring beans or classes to be created based on the active profile.

That is where the `@Profile` annotation is used.

43. What Problem Does `@Profile` Solve

Suppose we have a notification feature.

In development, we do not want to send real emails.

In production, we want to send real emails.

So we may create two implementations:

```
DevNotificationService → fake notification or console log  
ProdNotificationService → real notification or email service
```

Both classes can implement the same interface:

```
public interface NotificationService {  
    void sendNotification(String message);  
}
```

Development implementation:

```
@Service  
public class DevNotificationService implements NotificationService {  
  
    @Override  
    public void sendNotification(String message) {  
        System.out.println("DEV MODE: Notification not actually sent. Message: " + message);  
    }  
}
```

Production implementation:

```

@Service
public class ProdNotificationService implements
    NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sent. Message: " + message);
    }
}

```

Now both classes are marked as `@Service`.

So Spring creates both beans.

If we inject the interface:

```

@Autowired
private NotificationService notificationService;

```

Spring may get confused:

```

Which NotificationService bean should be injected?
DevNotificationService?
ProdNotificationService?

```

This can cause an error because multiple beans of the same type are available.

We need a way to say:

```

Create DevNotificationService only when dev profile is active.
Create ProdNotificationService only when prod profile is active.

```

That is exactly what `@Profile` does.

44. Basic Syntax of `@Profile`

```
@Profile("dev")
```

Meaning:

Create this bean only when dev profile is active.

Example:

```
@Service
@Profile("dev")
public class DevNotificationService implements NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("DEV MODE: Notification not actually sent. Message: " + message);
    }
}
```

Production version:

```
@Service
@Profile("prod")
public class ProdNotificationService implements NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sent. Message: " + message);
    }
}
```

Now run with dev:

```
java -jar app.jar --spring.profiles.active=dev
```

Result:

```
DevNotificationService bean is created.  
ProdNotificationService bean is not created.
```

Run with prod:

```
java -jar app.jar --spring.profiles.active=prod
```

Result:

```
ProdNotificationService bean is created.  
DevNotificationService bean is not created.
```

Simple understanding:

```
@Service → Spring, please create an object of this class.  
@Profile → But create it only when this profile is active.
```

45. Complete `@Profile` Demo

Interface:

```
public interface NotificationService {  
    void sendNotification(String message);  
}
```

Dev implementation:

```

@Service
@Profile("dev")
public class DevNotificationService implements NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("DEV MODE: Fake notification. Message: " + message);
    }
}

```

Prod implementation:

```

@Service
@Profile("prod")
public class ProdNotificationService implements NotificationService {

    @Override
    public void sendNotification(String message) {
        System.out.println("PROD MODE: Real notification sent. Message: " + message);
    }
}

```

Controller:

```

@RestController
@RequestMapping("/api/notification")
public class NotificationController {

    private final NotificationService notificationService;

    public NotificationController(NotificationService notificationService) {
        this.notificationService = notificationService;
    }

    @PostMapping
    public void sendNotification(String message) {
        notificationService.sendNotification(message);
    }
}

```

```

ationService) {
    this.notificationService = notificatio
}

@GetMapping("/send")
public String sendNotification() {
    notificationService.sendNotification("
application");
    return "Notification processed";
}
}

```

If `dev` is active, Spring injects `DevNotificationService`.

If `prod` is active, Spring injects `ProdNotificationService`.

The controller remains the same. The implementation changes based on the active profile.

46. When to Use Profile-Specific Files vs `@Profile`

This distinction is very important.

Use Profile-Specific Files When Values Change

Use profile-specific files when the class or logic remains the same, but configuration values change.

Examples:

```

Server port
Database URL
Database username
Database password
Logging level
Custom messages
Feature flags
JPA show-sql
Third-party API URLs

```

Example:

`application-dev.yml`:

```
server:
  port: 8081

app:
  message: Running in development
```

`application-prod.yml`:

```
server:
  port: 8080

app:
  message: Running in production
```

Here, the Java class does not change. Only the values change.

Use profile-specific files when your problem is:

I need different configuration values for different environments.

Use `@Profile` When Bean/Class Implementation Changes

Use `@Profile` when different environments need different implementations.

Examples:

```
DevEmailService vs ProdEmailService
DummyPaymentService vs RealPaymentService
LocalStorageService vs CloudStorageService
MockSmsService vs RealSmsService
FakeNotificationService vs RealNotificationService
```

Here, the difference is not just a value. The actual implementation is different.

Use `@Profile` when your problem is:

I need different beans/classes to be created for different environments

47. Quick Comparison

Topic	Use Case	Example
<code>application.properties</code>	Simple key-value configuration	<code>server.port=8081</code>
<code>application.yml</code>	Structured and nested configuration	<code>spring.datasource.url</code> grouped under <code>spring.datasource</code>
Profile-specific files	Different values for different environments	<code>application-dev.yml</code> , <code>application-prod.yml</code>
<code>spring.profiles.active</code>	Tells Spring Boot which profile to use	<code>dev</code> , <code>test</code> , <code>prod</code>
Command-line profile	Run same JAR with different profiles	<code>--spring.profiles.active=prod</code>
Environment variable profile	Activate profile from OS/server/deployment platform	<code>SPRING_PROFILES_ACTIVE=prod</code>
<code>@Value</code>	Read a single configuration value	<code>@Value("\${app.message}")</code>
<code>@Profile</code>	Create bean only for a specific profile	<code>@Profile("dev")</code>

48. Common Mistakes to Avoid

Mistake 1: Hardcoding Environment Values in Java Code

Avoid:

```
String dbUrl = "jdbc:mysql://prod-server:3306/student_db";
```

Prefer configuration files or environment variables.

Mistake 2: Keeping Both `.properties` and `.yml` Purpose

Avoid confusing project structure.

Prefer one consistent format.

Mistake 3: Hardcoding `prod` Profile Inside the JAR

Avoid:

```
spring:
  profiles:
    active: prod
```

inside the packaged application for production use.

Prefer passing the profile from outside using command line, environment variables, or deployment configuration.

Mistake 4: Using Too Many `@Value` Annotations for Complex Config

For a few values, `@Value` is fine.

For complex nested configuration, lists, or grouped values, prefer:

```
@ConfigurationProperties
```

Mistake 5: Confusing Profile Files With `@Profile`

Remember:

```
Profile-specific files → change values
@Profile                → change beans/classes
```

49. Final Mental Model

Use this simple mental model:

Configuration means values that can change without changing Java co

`application.properties` gives key-value configuration.

`application.yml` gives the same configuration in a cleaner nested structure.

Profiles allow the same application to run with different environment-specific values.

`@Profile` allows Spring to create different beans/classes based on the active environment.

The final goal is:

Same code
Same build
Different environment
Different configuration

50. Video Ending Summary

In this video, we learned:

Why configuration is needed
Why hardcoding is a bad practice
What `application.properties` is
How to read configuration using `@Value`
Why YAML is useful
How YAML indentation works
How to convert properties to YAML
How to represent lists in YAML
Why profiles are needed
How profile-specific files work
How to activate profiles

How to use command-line and environment variables
How @Profile controls bean creation
When to use profile files and when to use @Profile

The main takeaway is simple:

Spring Boot configuration helps us write flexible applications.
Profiles help the same application behave differently in different
@Profile helps us create environment-specific beans only when needed.